

Cuaderno de ejercicios adicionales — Cliente-Servidor

A. Sistema de chat privado

Desarrolla un cliente y servidor multihilo en Java. Los usuarios se autentican con usuario y contraseña. Una vez autenticados pueden enviar mensajes privados a otros usuarios. Los mensajes se almacenan en disco para el caso en que el destinatario no este conectado.

Ficheros del servidor: usuarios.txt | mensajes_usuario.txt

Aplicacion Cliente	Sentido	Aplicacion Servidor
Conexion al servidor	->	Acepta conexion: arranca hilo
Solicita usuario y contraseña por teclado. Envia "usuario:contrasena"	->	Comprueba en usuarios.txt. Envia "200 OK" o "ERROR". Si ERROR, cierra conexion.
Si 200 OK: envia "ENVIAR;destinatario;mensaje" -> o envia "LEER"		Recibe comando
	<-	Si ENVIAR: guarda en mensajes_destinatario.txt. Envia "200 OK" o "ERROR". Si LEER: cuenta lineas en mensajes_usuario.txt. Envia numero de mensajes.
Si LEER: recibe numero y luego los mensajes uno a uno y los muestra	<-	Envia mensajes linea a linea
Envia "FIN". Cierra conexion.	->	Cierra conexion

Ampliacion: permite que el cliente repita el bucle (ENVIAR/LEER) hasta que envíe FIN.

B. Cajero automatico remoto

Servidor multihilo que gestiona cuentas bancarias. El cliente se autentica con numero de cuenta y PIN. Puede consultar saldo, ingresar o retirar dinero. El servidor actualiza el fichero de cuentas tras cada operacion.

Ficheros del servidor: cuentas.txt (formato: cuenta;PIN;saldo)

Aplicacion Cliente	Sentido	Aplicacion Servidor
Conexion al servidor	->	Acepta conexion: arranca hilo
Solicita cuenta y PIN. Envia "cuenta;PIN"	->	Busca en cuentas.txt. Envia "200 OK" o "ERROR". Si ERROR, cierra conexion.
Muestra menu: 1-Saldo, 2-Ingresar, 3-Retirar. Envia opcion elegida.	->	Recibe opcion
	<-	Si opcion 1: envia saldo actual. Si opcion 2 o 3: envia "IMPORTE" solicitando cantidad.
Si recibe "IMPORTE": pide cantidad por teclado y la envia.	->	Actualiza cuentas.txt. Si saldo insuficiente en retirada: envia "ERROR_SALDO". Si OK: envia nuevo saldo.
Muestra resultado. Envia "FIN". Cierra conexion.	->	Cierra conexion

Ampliacion: permite que el cliente realice varias operaciones en la misma sesion antes de enviar FIN.

C. Sistema de votacion

El servidor gestiona una votacion con varias opciones. El cliente se identifica con su DNI (solo puede votar una vez). Envia su voto y puede consultar el recuento provisional.

Ficheros del servidor: votantes.txt | votos.txt | opciones.txt

Aplicacion Cliente	Sentido	Aplicacion Servidor
Conexion al servidor	->	Acepta conexion: arranca hilo
Solicita DNI por teclado. Envia DNI.	->	Comprueba en votantes.txt si ya voto. Envia "PUEDE_VOTAR" o "YA_VOTADO".
Recibe respuesta. Si YA_VOTADO: cierra conexion.	<-	
Envia "GET_OPCIONES"	->	Lee opciones.txt. Envia numero de opciones, luego las opciones una a una.
Muestra opciones. Solicita voto. Envia "VOTO;opcion".	->	Registra en votos.txt. Aniade DNI a votantes.txt. Envia "200 OK" o "ERROR".
Si quiere ver resultados: envia "RESULTADOS". Si no: envia "FIN".	->	Si RESULTADOS: cuenta votos por opcion y los envia. Cierra conexion.
Muestra resultados. Cierra conexion.	<-	

Ampliacion: sincroniza el acceso a los ficheros con bloques synchronized para evitar condiciones de carrera entre hilos concurrentes.

D. Sistema de reservas

Servidor multihilo para gestionar reservas de plazas (aula, sala, evento). El cliente se autentica, consulta la disponibilidad y realiza o cancela una reserva. El servidor mantiene el estado en un fichero.

Ficheros del servidor: usuarios.txt | reservas.txt (formato: fecha;plaza;usuario)

Aplicacion Cliente	Sentido	Aplicacion Servidor
Conexion al servidor	->	Acepta conexion: arranca hilo
Envia "usuario;contrasena"	->	Comprueba en usuarios.txt. Envia "200 OK" o "401". Si 401, cierra conexion.
Envia comando: "DISPONIBILIDAD" o "RESERVAR;fecha;plaza" o "CANCELAR;fecha;plaza"	->	Recibe comando
	<-	Si DISPONIBILIDAD: lee reservas.txt y envia plazas libres. Si RESERVAR: comprueba disponibilidad. Si OK guarda y envia "200 OK". Si CANCELAR: elimina reserva. Envia "200 OK" o "ERROR".
Muestra respuesta. Envia "FIN". Cierra conexion. ->		Cierra conexion

Ampliacion: permite al cliente realizar varias operaciones en bucle hasta enviar FIN.

E. Monitor de temperatura

El cliente envia lecturas de temperatura desde un sensor (simulado por teclado). El servidor las almacena con timestamp, calcula la media y lanza una alerta por consola si alguna lectura supera el umbral configurado.

Ficheros del servidor: config.txt (formato: umbral=35.5) | lecturas_<sensor>.txt

Aplicacion Cliente	Sentido	Aplicacion Servidor
Conexion al servidor	->	Acepta conexion: arranca hilo. Lee umbral desde config.txt.
Envia identificador de sensor (string)	->	Recibe ID de sensor. Crea lecturas_sensor.txt si no existe.
Solicita numero de lecturas a enviar. Envia numero.	->	Recibe numero. Envia "PREPARADO".
Envia lecturas una a una (valores decimales)	->	Recibe cada lectura. Guarda con timestamp. Si supera umbral: muestra alerta por consola.
Envia "FIN"	->	Calcula media de todas las lecturas recibidas. Envia media al cliente.
Recibe y muestra la media. Cierra conexion.	<-	Cierra conexion

Formato lecturas_sensor.txt: timestamp;valor — una lectura por linea.